DOSSIER PROJET

Titre Professionnel Développeur Web et Web Mobile

Projet : Miftah Arabi

Plateforme pédagogique d'apprentissage de la langue arabe

Présenté par : Zhaki El Ouarroudi

Centre de formation : ALT RH Formation

Session : 2026

Projet réalisé dans le cadre de la formation DWWM

Avril 2026

Sommaire

1. Présentation du projet 5
1.1 Contexte et origine du projet 5
1.2 Présentation générale 5
1.3 Stack technique 6
2. Étude de l'existant 6
2.1 Duolingo 6
2.2 Madrasa.co 7
2.3 YouTube 7
2.4 Positionnement de Miftah Arabi 7
3. Expression des besoins 7
3.1 Recueil des besoins 7
3.2 User Stories 8
3.3 Fonctionnalités hors périmètre 9
4. Gestion de projet 9
4.1 Méthodologie 9
4.2 Versioning Git et GitHub 10
4.3 Déroulement des sprints 11
4.4 Outils utilisés pendant le développement 12
5. Conception 13
5.1 Conception fonctionnelle 13
5.2 Wireframes et maquettes 13
5.3 Base de données 15
5.4 Architecture applicative 17
5.5 Sauvegarde et restauration 17
6. Charte graphique 18
6.1 Identité visuelle 18
6.2 Couleurs 18
6.3 Typographie et lisibilité 19

6.4 Front-office et back-office 19
7. Réalisations 20
7.1 Espace visiteur 20
7.2 Authentification et espace utilisateur 21
7.3 Espace administration 23
7.4 Mode maintenance 25
7.5 SEO et pages annexes 25
8. Sécurité 27
8.1 Contrôle des accès 27
8.2 Sécurité des mots de passe 28
8.3 Vérification d'email 28
8.4 Protection CSRF 28
8.5 Protection contre les injections 28
8.6 Sécurisations des uploads 28
8.7 Limitation des tentatives de connexion 28
8.8 Maintenance et administration 28
9. Veille technologique 29
9.1 Sources utilisées 29
9.2 Exemples de sujets suivis 29
9.3 Utilité concrète de la veille 29
10. Jeu d'essai 29
10.1 données de test utilisées 30
10.2 Scénarios testés 30
11. Difficultés rencontrées 31
11.1 Vérification d'email 31
11.2 Upload d'images 31
11.3 Sécurité Git et fichiers sensibles 31
11.4 Évolution du périmètre 31
11.5 Écart entre théorie et production 32
11.6 Correctifs de fiabilisation apportés en phase finale 32
12. Bilan et perspectives 32
12.1 Bilan 32

12.2 Ce que j'ai appris 33
12.3 Ce que j'améliorerais 33
12.4 Perspectives 34
13. Annexes 34
Annexe A - Compétences DWWM mobilisées 34
Annexe B - Structure du projet 34
Annexe C - Captures techniques complémentaires 35

1. Présentation du projet

1.1 Contexte et origine du projet

J'ai développé **Miftah Arabi** ("La clé pour apprendre l'arabe") dans le cadre de ma formation **Développeur Web et Web Mobile (DWWM)** à ALT RH Formation.

Ce projet a été réalisé pendant ma formation, en m'appuyant à la fois sur les **cours en présentiel** et sur la **playlist vidéo** mise à disposition par notre formateur. Cette organisation m'a beaucoup aidé, car elle me permettait de revoir certaines étapes chez moi, à mon rythme, quand je bloquais sur un point technique ou quand j'avais besoin de refaire une manipulation plusieurs fois pour bien la comprendre.

Le projet est né d'un besoin concret exprimé par un enseignant de langue arabe. Ses contenus pédagogiques existaient déjà, mais ils étaient dispersés entre plusieurs plateformes comme YouTube, TikTok, Instagram et Telegram. Cette dispersion rendait l'accès moins lisible pour les apprenants et compliquait l'organisation des ressources.

L'objectif initial était donc de créer une plateforme web claire, structurée et évolutive permettant de centraliser ces contenus, de les organiser par catégories, niveaux et mots-clés, puis de les administrer simplement depuis un back-office.

Ce projet m'a permis de mettre en pratique les compétences attendues dans le titre professionnel DWWM : analyse du besoin, conception de base de données, développement front-end et back-end sécurisé, gestion de projet, qualité du code, documentation et préparation du déploiement. C'est un projet sur lequel j'ai vraiment progressé, mais aussi rencontré de vraies difficultés par moments, notamment sur l'authentification, la sécurité, les relations entre entités et la mise en place de certaines fonctionnalités d'administration.

1.2 Présentation générale

Miftah Arabi est une plateforme web pédagogique destinée à un public francophone souhaitant apprendre la langue arabe. Le site propose des leçons structurées, accompagnées de ressources vidéo, avec plusieurs parcours d'usage :

- Un espace visiteur pour consulter les leçons publiées ;
- Un espace utilisateur pour gérer son compte et ses favoris ;
- Un espace d'administration pour gérer le contenu pédagogique et les utilisateurs.

Le projet a d'abord été pensé comme un **MVP (Minimum Viable Product).** Au départ, je visais surtout une première version fonctionnelle autour de la consultation des leçons et de leur gestion côté administration. En avançant dans le projet, j'ai ajouté d'autres fonctionnalités utiles à l'expérience utilisateur et à la sécurité : inscription, vérification d'email, connexion sécurisée, réinitialisation du mot de passe, gestion des favoris, mode maintenance et pages légales.

Comme souvent dans un projet de formation, tout n'a pas été linéaire. J'ai dû revenir plusieurs fois sur des parties déjà faites, corriger mes choix, relire mes cours, revoir des

vidéos du formateur et refaire certaines étapes pour arriver à un résultat propre et présentable.

Dépôt GitHub : https://github.com/E-Zhaki/projet-miftah-arabi

Lien du site en production : https://palegoldenrod-snail-949194.hostingersite.com/

1.3 Stack technique

Pour ce projet, j'ai choisi les technologies suivantes :

- **Back-end :** PHP 8.2, Symfony 7.4, Doctrine ORM
- **Base de données :** MySQL 8
- **Front-end :** Twig, HTML, SCSS, Bootstrap 5.3, JavaScript
- **Build front-end :** Webpack Encore
- **Bundles principaux :** VichUploaderBundle, KnpPaginatorBundle, LiipImagineBundle, SymfonyCasts VerifyEmailBundle, SymfonyCasts ResetPasswordBundle, StofDoctrineExtensionsBundle
- **Outils de qualité :** PHP CS Fixer, PHPStan, linters Symfony
- **Versioning :** Git et GitHub
- **Gestion de projet :** Trello, méthode Agile Scrum

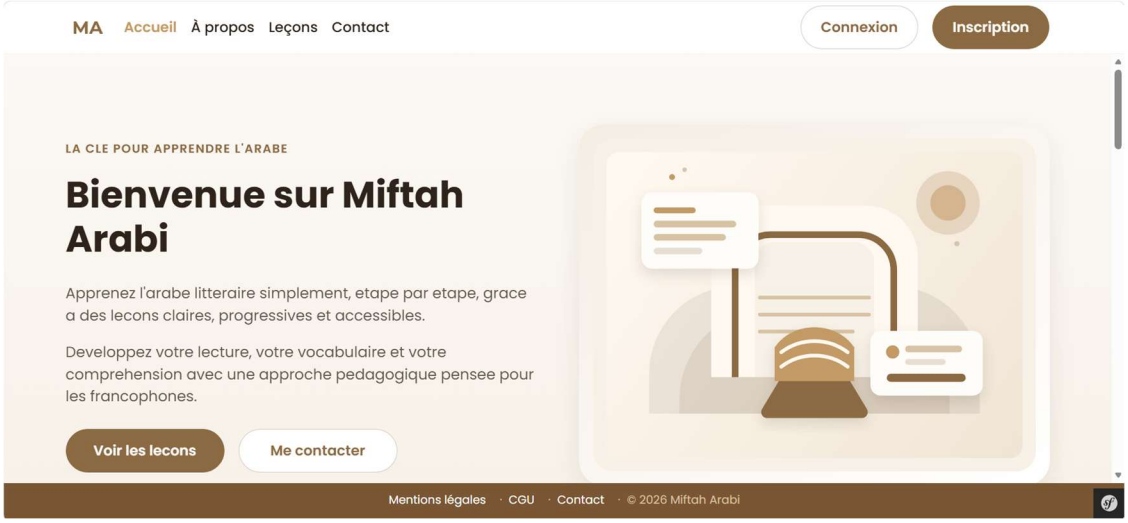


Figure 1 : Page d'accueil de Miftah Arabi

2. Étude de l'existant

Avant de commencer le développement, j'ai étudié plusieurs plateformes déjà présentes sur le marché afin d'identifier leurs forces, leurs limites et la place que pouvait prendre Miftah Arabi.

2.1 Duolingo

Duolingo propose un apprentissage de l'arabe sous forme de micro-exercices et de gamification. La plateforme est accessible et très connue, mais elle reste entièrement

généraliste. Un enseignant indépendant ne peut pas y publier librement ses propres contenus ni structurer sa pédagogie selon ses besoins.

2.2 Madrasa.co

Madrasa propose des cours d'arabe structurés, souvent orientés vers l'arabe coranique. Le contenu est de qualité, mais l'enseignant extérieur ne dispose pas d'un espace simple pour publier et gérer ses propres leçons.

2.3 YouTube

YouTube est très utilisé par les enseignants pour diffuser des vidéos pédagogiques. C'était d'ailleurs déjà le cas du commanditaire. En revanche, YouTube ne permet pas d'offrir une vraie organisation pédagogique : pas de catégorisation métier, pas de structure par niveaux, pas d'espace d'administration dédié au projet, pas de système de favoris personnalisé au contexte du site.

2.4 Positionnement de Miftah Arabi

Le positionnement de Miftah Arabi est celui d'une **plateforme sur mesure** pour un enseignant indépendant :

- Le contenu appartient au commanditaire ;
- L'organisation pédagogique est libre ;
- Les ressources YouTube sont reliées aux leçons du site ;
- L'administration du contenu est centralisée ;
- L'expérience de consultation est pensée pour un public francophone débutant ou intermédiaire.

Plateforme	Contenu personnalisable	Espace admin	Ressources vidéo liées	Favoris
Duolingo	Non	Non	Non	Non
Madrasa	Non	Non	Oui	Non
YouTube	Partiel	Non	Oui	Oui
Miftah Arabi	**Oui**	**Oui**	**Oui**	**Oui**

3. Expression des besoins

3.1 Recueil des besoins

Le recueil des besoins a été réalisé à partir d'un échange initial entre le client et le développeur. Plusieurs attentes fortes sont apparues :

- Centraliser des contenus pédagogiques déjà existants ;
- Proposer des cours gratuits accessibles sans inscription ;
- Organiser les leçons par catégories et niveaux ;
- Associer des ressources vidéo à chaque leçon ;

- Disposer d'un espace d'administration simple et sécurisé ;
- Donner une image sérieuse, pédagogique et accueillante.

Le client souhaitait également une approche **MVP**, c'est-à-dire une première version fonctionnelle centrée sur les besoins essentiels, avec possibilité d'évolution progressive.

3.2 User Stories

Pour traduire les besoins en fonctionnalités concrètes, j'ai rédigé des user stories.

Visiteur

- En tant que visiteur, je souhaite voir les dernières leçons publiées sur la page d'accueil afin d'avoir un aperçu du contenu disponible.
- En tant que visiteur, je souhaite consulter la liste complète des leçons avec filtrage par catégorie, niveau ou tag afin de trouver rapidement le contenu qui m'intéresse.
- En tant que visiteur, je souhaite accéder au détail d'une leçon et à ses ressources associées afin de suivre le cours complet.
- En tant que visiteur, je souhaite utiliser un formulaire de contact afin de poser une question au responsable du site.
- En tant que visiteur, je souhaite créer un compte afin d'accéder aux fonctionnalités réservées aux utilisateurs connectés.

Utilisateur connecté

- En tant qu'utilisateur connecté, je souhaite ajouter une leçon à mes favoris afin de la retrouver plus tard.
- En tant qu'utilisateur connecté, je souhaite retirer une leçon de mes favoris afin de garder une liste pertinente.
- En tant qu'utilisateur connecté, je souhaite modifier mes informations de profil afin de maintenir mon compte à jour.
- En tant qu'utilisateur connecté, je souhaite changer mon mot de passe ou le réinitialiser par email si je l'oublie.

Administrateur

- En tant qu'administrateur, je souhaite accéder à un tableau de bord afin d'avoir une vision globale du site.
- En tant qu'administrateur, je souhaite gérer les catégories, les tags, les leçons et les ressources afin d'organiser le contenu pédagogique.
- En tant qu'administrateur, je souhaite publier ou dépublier une leçon afin de contrôler ce qui est visible côté public.
- En tant qu'administrateur, je souhaite consulter les messages de contact afin d'assurer le suivi des demandes.
- En tant qu'administrateur, je souhaite gérer les utilisateurs et leurs rôles afin de contrôler les accès.

- En tant qu'administrateur, je souhaite activer un mode maintenance afin d'intervenir sur le site sans exposition publique.

3.3 Fonctionnalités hors périmètre

Certaines fonctionnalités ont été identifiées puis volontairement exclues du périmètre du MVP :

- Suivi de progression individuelle ;
- Quiz ou exercices interactifs ;
- Commentaires sur les leçons ;
- Paiements et abonnements ;
- Application mobile native.

Ce choix m'a permis de garder un périmètre réaliste et de livrer une première version stable, maintenable et cohérente.

---

4. Gestion de projet

4.1 Méthodologie

Je me suis appuyé sur une logique **Agile Scrum** pour organiser le projet. Je ne prétends pas avoir appliqué Scrum de façon parfaite comme dans une équipe senior, mais le fait de découper le travail en sprints et en tickets m'a vraiment aidé à avancer sans me perdre.

Le développement a été découpé en sprints avec priorisation des fonctionnalités les plus importantes. Cette organisation m'a permis de livrer progressivement des blocs cohérents, de contrôler l'avancement et d'ajuster le périmètre quand cela était nécessaire.

L'outil **Trello** a servi à organiser les tâches, visualiser leur état d'avancement et structurer le backlog. Dans les faits, c'était un vrai repère de travail pour savoir ce qui était terminé, ce qui était en cours et ce qu'il fallait encore corriger.

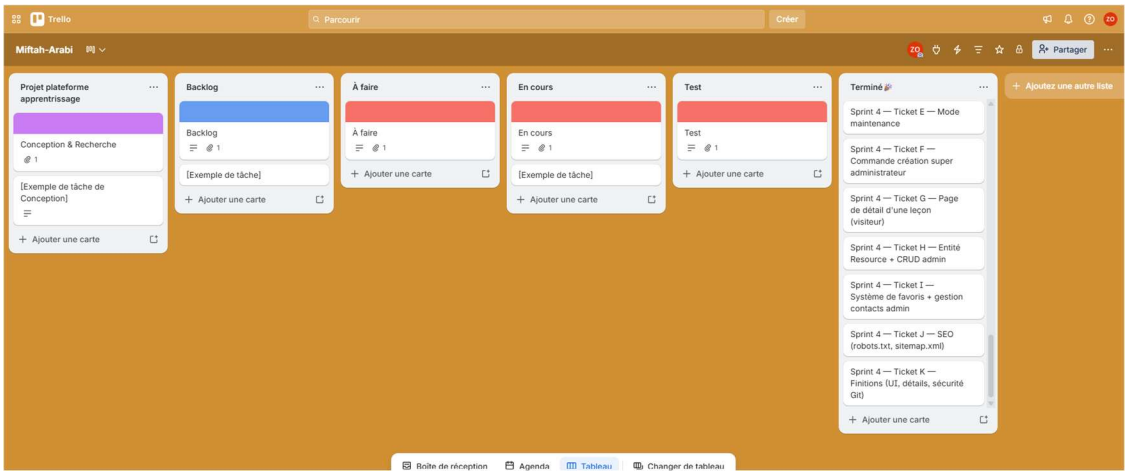


Figure 2 : Tableau Trello - organisation des tâches du projet

4.2 Versioning Git et GitHub

Le projet a été versionné avec **Git** et hébergé sur **GitHub**. L'analyse du dépôt montre une organisation par branches de fonctionnalité, avec fusion progressive sur main.

Éléments observés dans le dépôt :

- **87 commits** sur la branche principale ;
- **28 pull requests fusionnées** ;
- **29 branches de sprint/ticket** nommées de façon explicite ;
- Un dépôt distant : https://github.com/E-Zhaki/projet-miftah-arabi.git.

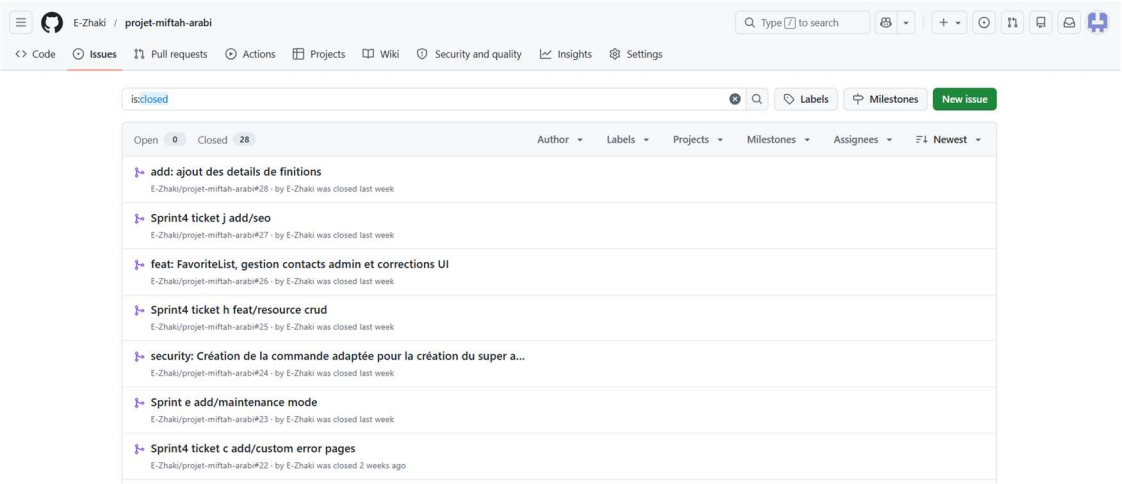
Les branches suivent une convention de nommage du type :

sprintX-ticketY-type/feature-name

Exemples réels observés :

- Sprint1-ticketC-add/users-table
- Sprint2-ticketA-feat/registration-system
- Sprint2-ticketE-feat/reset-password
- Sprint3-ticketC-feat/lessons-management
- Sprint4-ticketI-feat/favorite-list-and-contact-admin
- Sprint4-ticketJ-add/seo

Cette organisation constitue une vraie preuve de gestion de projet et de progression structurée.



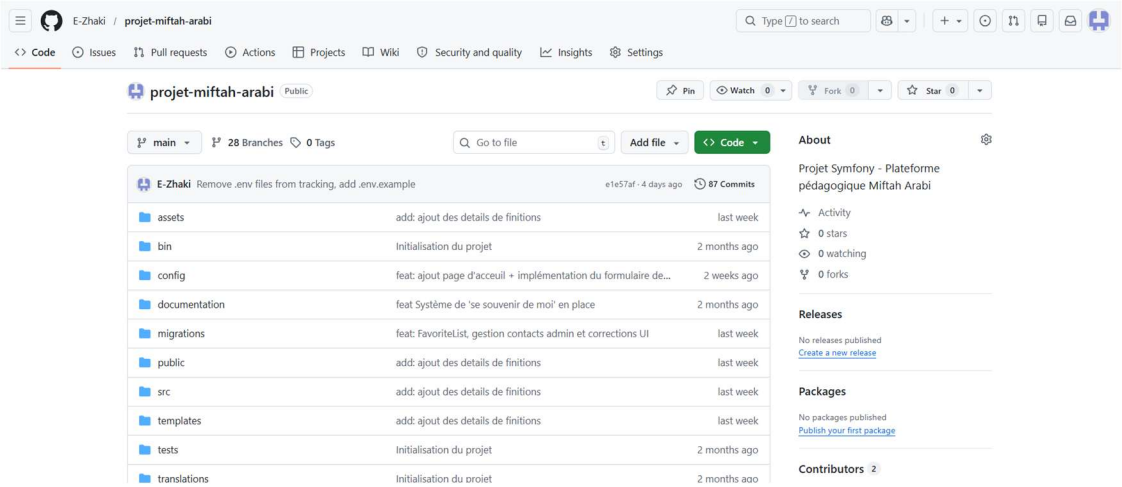


Figure 3 : Historique GitHub - branches et pull requests du projet

4.3 Déroulement des sprints

L'historique Git permet de reconstituer une progression logique du projet.

Sprint 1 - Fondations

- Initialisation du projet Symfony ;
- Installation des outils de qualité ;
- Création de la table User ;
- Mise en place de Webpack Encore, Sass et Bootstrap ;
- Création des premières pages visiteur.

Sprint 2 - Authentification

- Inscription ;
- Connexion ;
- Limitation des tentatives de connexion ;
- Fonctionnalité "se souvenir de moi" ;
- Réinitialisation du mot de passe ;
- Vérification du statut de compte avant connexion.

Sprint 3 - Back-office et gestion du contenu

- Sécurisation des espaces utilisateur et administrateur ;
- CRUD des catégories ;
- CRUD des leçons ;
- CRUD des tags ;
- Relation many-to-many leçons/tags ;
- Gestion du profil ;
- Gestion des rôles utilisateurs.

Sprint 4 - Finalisation fonctionnelle

- Affichage des leçons et filtres ;
- Page d'accueil ;
- Pages d'erreur personnalisées ;
- Tableau de bord ;
- Mode maintenance ;
- Commande de création du super administrateur ;
- Page de détail d'une leçon ;
- CRUD des ressources ;
- Système de favoris ;
- Gestion des messages de contact ;
- SEO et finitions.

4.4 Outils utilisés pendant le développement

Les outils suivants ont rythmé le projet :

- **Trello** pour la gestion du backlog ;
- **Git/GitHub** pour le versioning ;
- **Mailtrap** pour les tests d'emails en développement ;
- **Balsamiq** pour les wireframes ;
- **DrawSQL** pour la modélisation de la base ;
- **WampServer / phpMyAdmin** pour l'environnement local et la base de données.

Au quotidien, j'ai alterné entre les cours en centre, le travail personnel et le revisionnage de la playlist du formateur. Cela m'a permis de consolider les notions vues en présentiel et de mieux comprendre certaines étapes de développement en les refaisant moi-même.

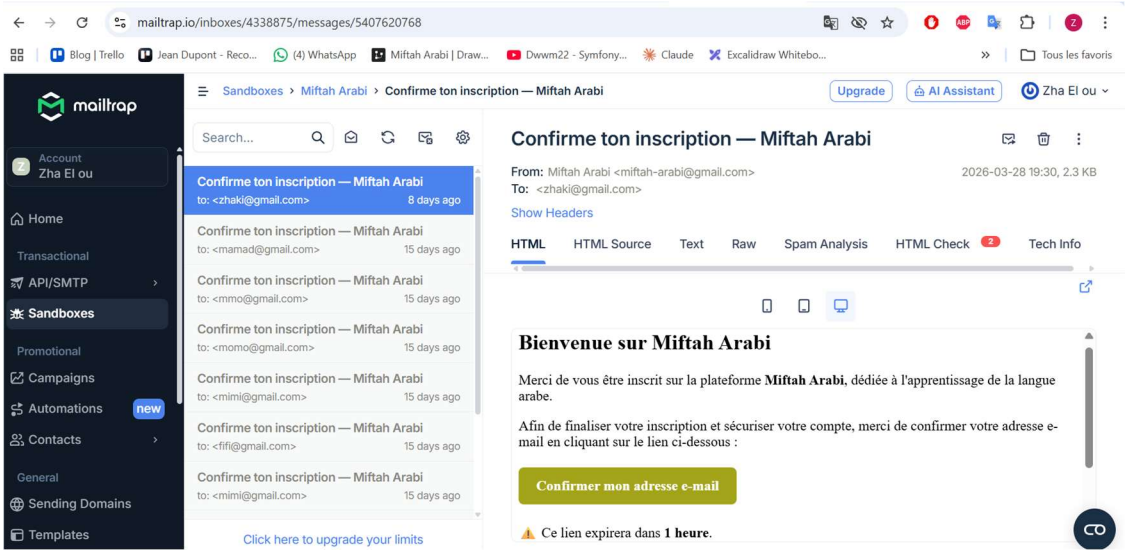


Figure 4 : Mailtrap - test des emails de confirmation et de réinitialisation

5. Conception

5.1 Conception fonctionnelle

La conception du projet s'est appuyée sur plusieurs documents préparatoires :

- Expression et recueil des besoins ;
- Analyse initiale ;
- Étude de faisabilité ;
- Cahier des charges ;
- Validation et approbation du cahier des charges ;
- Spécifications fonctionnelles ;
- Spécifications techniques et workflow.

Je n'ai pas codé le projet directement sans préparation. J'ai d'abord produit plusieurs brouillons de conception, puis j'ai ajusté ces documents au fur et à mesure que le projet évoluait. Cette phase m'a aidé à mieux comprendre ce que je voulais faire, même si tout n'était pas parfaitement défini dès le départ.

Cette documentation m'a permis de cadrer le MVP avant le développement, de formaliser les attentes du client et de traduire les besoins en fonctionnalités concrètes.

Ces documents n'étaient pas tous destinés à être rendus comme livrables indépendants au jury, mais ils ont joué un vrai rôle dans mon organisation. Ils m'ont servi de fil conducteur entre le besoin de départ, la conception du projet, les choix techniques et le développement réel de l'application.

5.2 Wireframes et maquettes

Avant de développer les interfaces, j'ai réalisé des wireframes avec **Balsamiq**. L'objectif n'était pas encore de produire un design final, mais de valider :

- La hiérarchie des contenus ;
- La navigation ;
- Les parcours utilisateur ;
- La répartition des blocs fonctionnels selon les pages.

Les maquettes principales concernent :

- La page d'accueil ;
- La liste des leçons ;
- La page de détail d'une leçon ;
- La page de connexion ;
- La page profil ;
- Le tableau de bord administrateur ;
- Le formulaire de gestion des leçons.

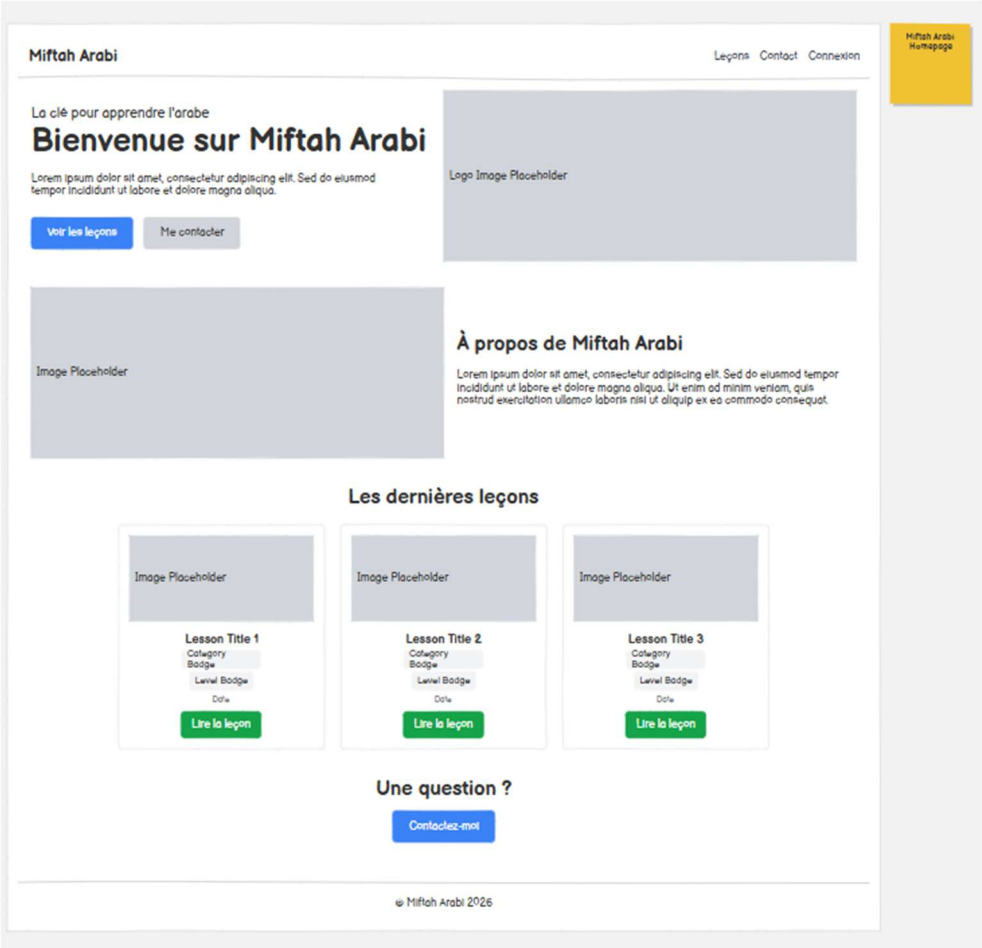


Figure 5 : Wireframe - page d'accueil

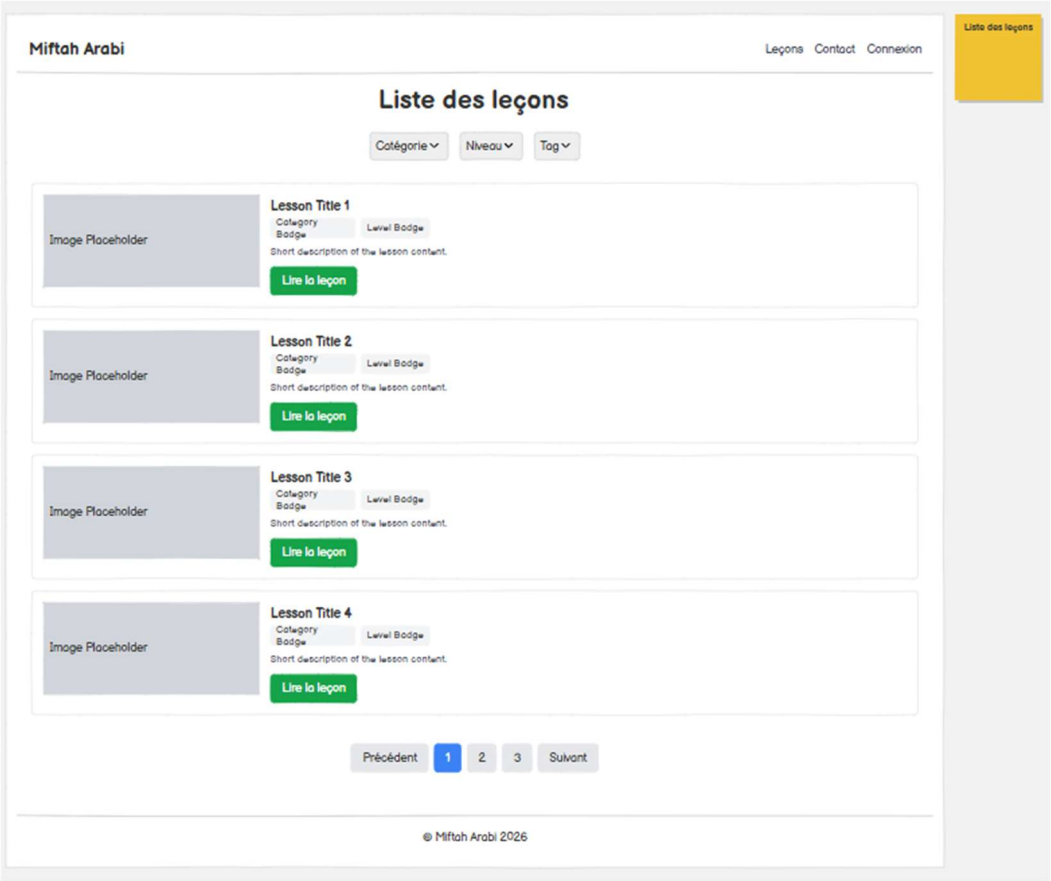


Figure 6 : Wireframe - liste des leçons

Figure 7 : Wireframe - tableau de bord administrateur

5.3 Base de données

La base de données a été modélisée avant développement, puis implémentée avec Doctrine.

Le projet repose sur **8 entités** :

- User
- Lesson
- Category
- Tag
- Resource
- FavoriteList
- ContactMessage
- ResetPasswordRequest

Les principales relations sont :

- Une Category possède plusieurs Lesson ;
- Une Lesson possède plusieurs Resource ;
- Une Lesson peut être reliée à plusieurs Tag et inversement ;
- Un User peut posséder plusieurs favoris via FavoriteList ;
- ResetPasswordRequest est reliée à User.

Cette modélisation répond à la logique métier du projet :

- Une leçon est l'unité pédagogique principale ;
- Une catégorie structure les thèmes ;
- Un tag apporte une indexation transversale ;
- Une ressource enrichit la leçon ;
- Un favori matérialise la relation utilisateur / leçon.

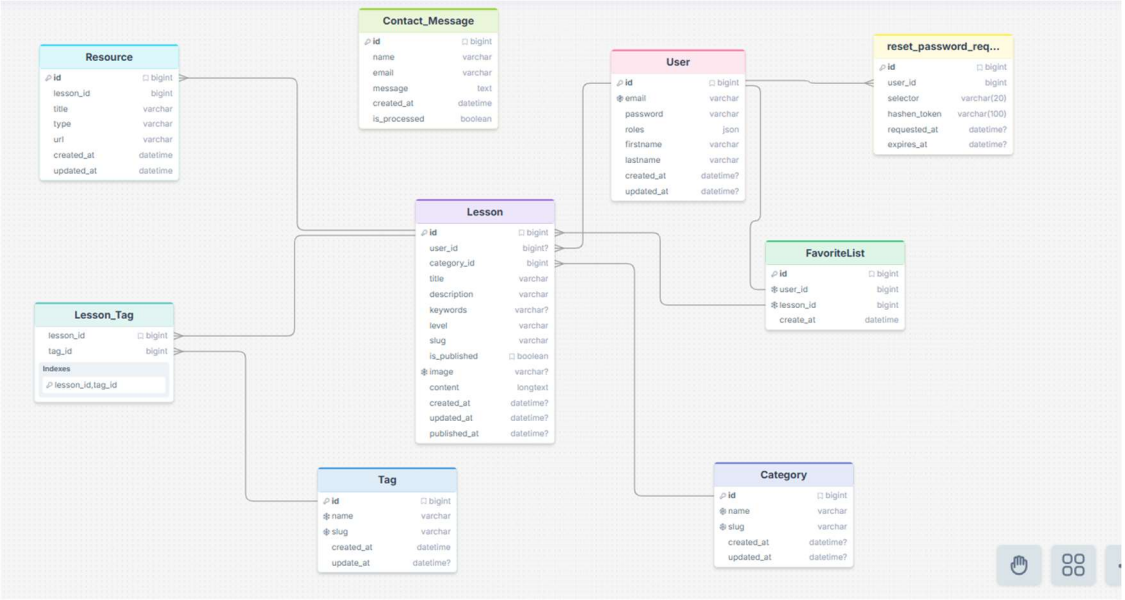


Figure 8 : Schéma de base de données DrawSQL

5.4 Architecture applicative

Le projet suit une architecture **MVC** conforme aux bonnes pratiques Symfony :

- **Model :** entités Doctrine, repositories, logique métier ;
- **View :** templates Twig ;
- **Controller :** gestion des requêtes HTTP, des formulaires et des flux utilisateur.

À la date de cette analyse, l'architecture comprend :

- **20 contrôleurs**
- **8 entités**
- **11 FormType**
- **8 repositories**
- **11 migrations**
- **4 commandes Symfony** (GenerateAppLocalSecretKeyCommand, MaintenanceModeCommand, CreateSuperAdminCommand, SeedDemoContentCommand)
- **65 déclarations de routes par attribut** dans les contrôleurs

Cette structure montre un projet organisé, cohérent et exploitable.

5.5 Sauvegarde et restauration

En développement, la sauvegarde des données peut être réalisée par export SQL depuis phpMyAdmin. En complément, la structure de la base est aussi versionnée via les migrations Doctrine, ce qui permet de reconstruire le schéma de base à partir du projet.

Le site a été mis en production sur un hébergement mutualisé Hostinger. Les étapes réellement effectuées sont les suivantes :

• Connexion au serveur en SSH et accès au dossier public_html ;

• Vérification de la version PHP (8.2) et installation de Composer puis de Node.js via NVM ;

• Clonage du dépôt Git du projet sur le serveur ;

• Installation des dépendances : composer install et npm install ;

• Compilation des assets en mode production : npm run build ;

• Création du fichier .env.local avec les vraies variables d'environnement ;

• Exécution des migrations Doctrine : php bin/console doctrine:migrations:migrate ;

• Création du compte super-administrateur via la commande dédiée ;

• Configuration du fichier .htaccess à la racine pour rediriger vers le dossier public/ ;

• Installation du package symfony/apache-pack pour générer le .htaccess de public/ ;

• Vidage du cache Symfony : php bin/console cache:clear ;

• Configuration des permissions Linux : 755 sur l'arborescence, 775 sur var/ et public/ ;

• Activation du certificat SSL (HTTPS) côté hébergeur.

6. Charte graphique

6.1 Identité visuelle

J'ai souhaité proposer une interface sobre, lisible et chaleureuse, cohérente avec l'univers de l'apprentissage de l'arabe. L'objectif n'était pas de produire une interface surchargée, mais un site pédagogique accessible à un public débutant.

6.2 Couleurs

La palette repose principalement sur :

- Des tons chauds, bruns et sable ;
- Des accents or et beige ;
- Un fond clair favorisant la lecture ;
- Des contrastes suffisants pour conserver une bonne lisibilité.

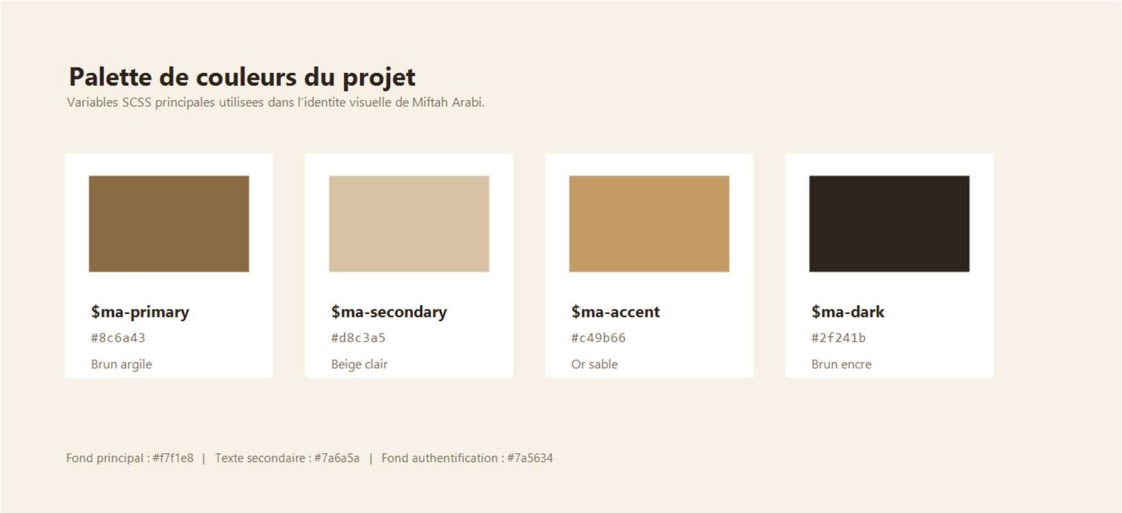


Figure 9 : Palette de couleurs utilisée dans le projet

6.3 Typographie et lisibilité

L'accent a été mis sur :

- Une hiérarchie visuelle claire ;
- Des titres lisibles ;
- Des espacements suffisants ;
- Une lecture confortable sur desktop comme sur mobile.

6.4 Front-office et back-office

J'ai distingué visuellement les espaces :

- **Front-office :** interface plus travaillée, plus chaleureuse et orientée pédagogie ;
- **Back-office :** interface plus sobre, fonctionnelle et centrée sur la productivité.

Front-office | Back-office

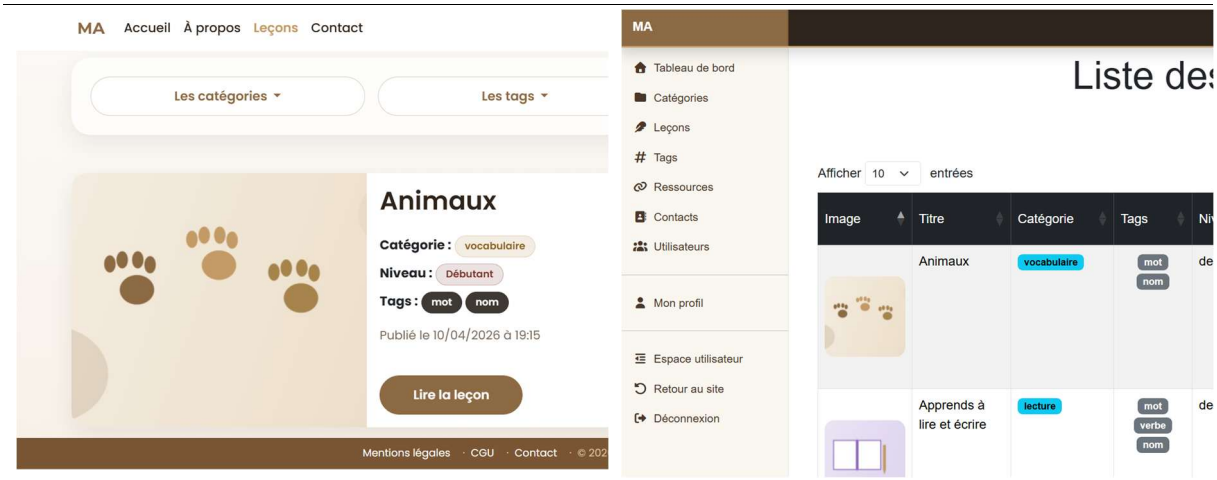


Figure 10 : Comparaison visuelle entre la liste des leçons côté visiteur et côté administration

7. Réalisations

7.1 Espace visiteur

L'espace visiteur permet :

- De consulter la page d'accueil ;
- D'afficher les dernières leçons publiées ;
- D'accéder à la liste complète des leçons ;
- De filtrer les leçons par catégorie, tag et niveau ;
- De consulter le détail d'une leçon et ses ressources ;
- D'utiliser le formulaire de contact ;
- D'accéder aux pages légales et au plan du site.

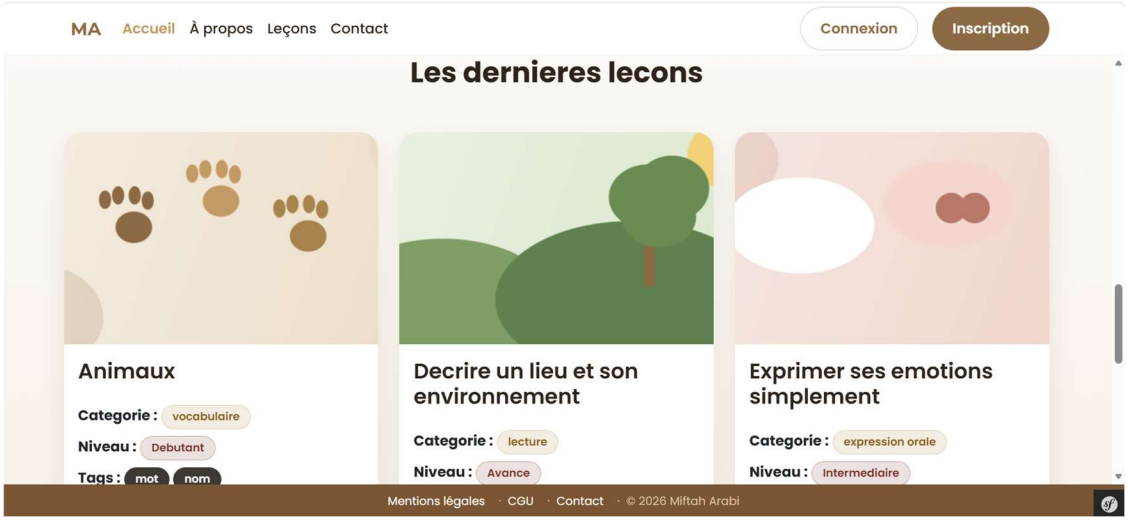


Figure 11 : Page d'accueil du site

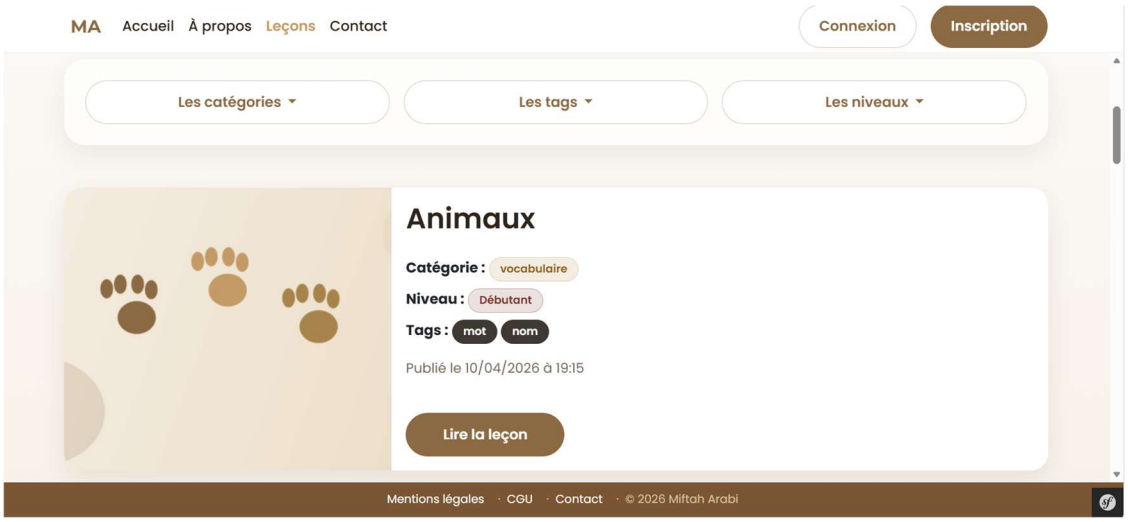


Figure 12 : Liste des leçons avec filtres et pagination

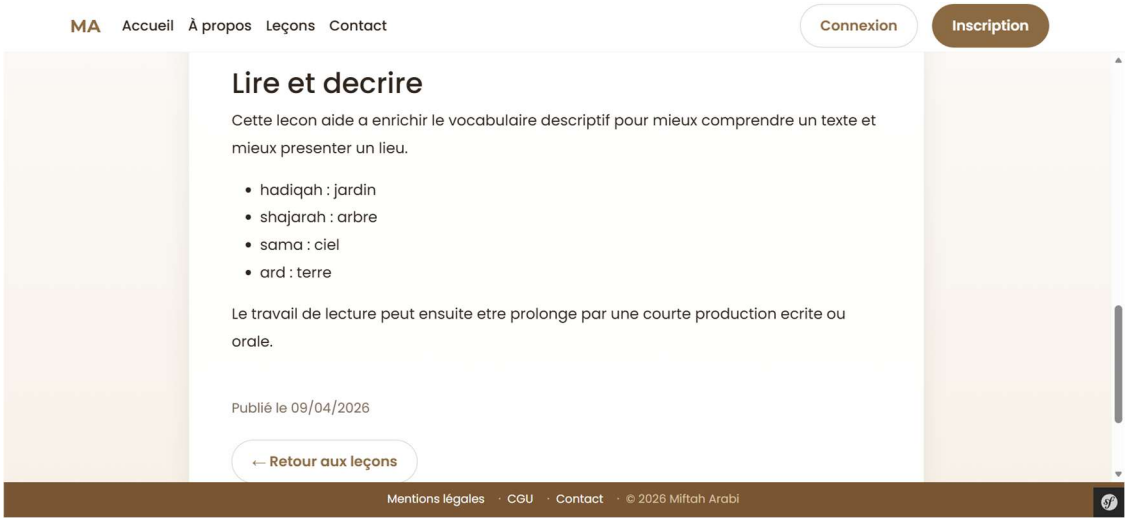


Figure 13 : Détail d'une leçon et ressources associées

7.2 Authentification et espace utilisateur

Le projet comprend un système d'authentification complet :

- Inscription ;
- Vérification d'email ;
- Connexion ;
- Limitation des tentatives de connexion ;
- Fonctionnalité "se souvenir de moi" ;
- Réinitialisation du mot de passe ;
- Gestion du profil ;
- Gestion des favoris.

Le UserChecker empêche la connexion tant que le compte n'a pas été vérifié par email.

Figure 14 : Formulaire d'inscription

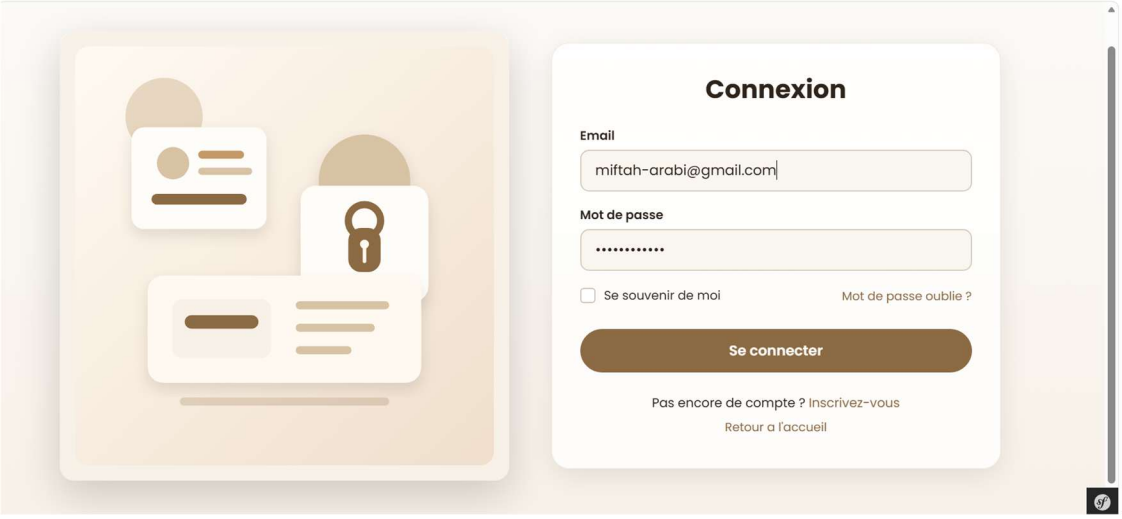


Figure 15 : Page de connexion

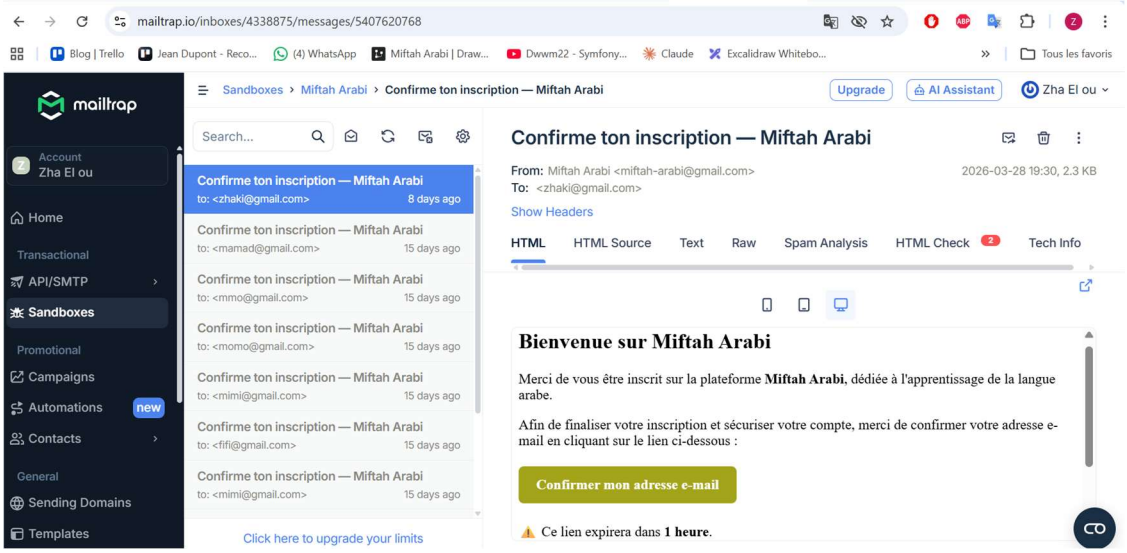


Figure 16 : Email de vérification reçu dans Mailtrap

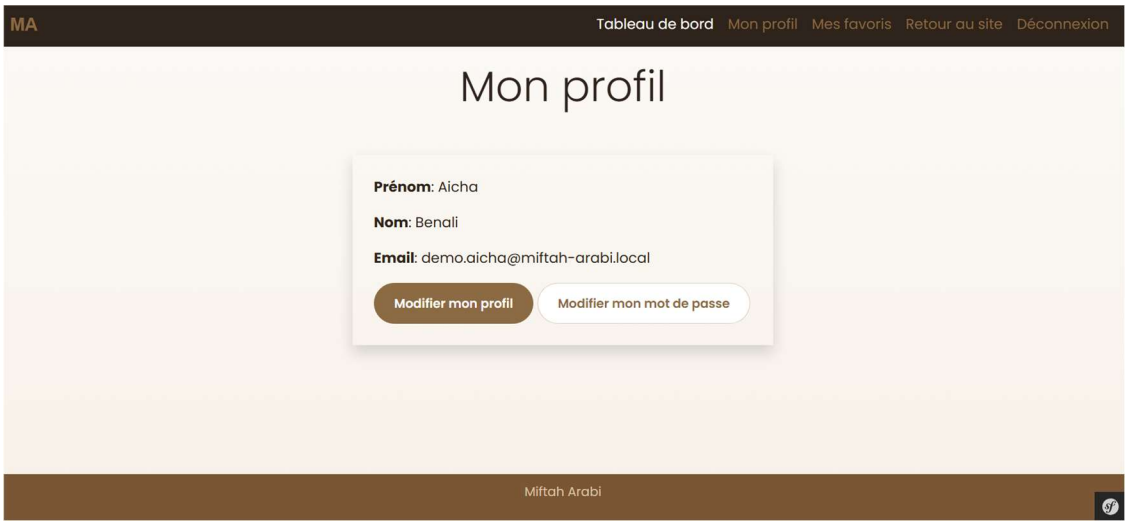


Figure 17 : Espace profil utilisateur

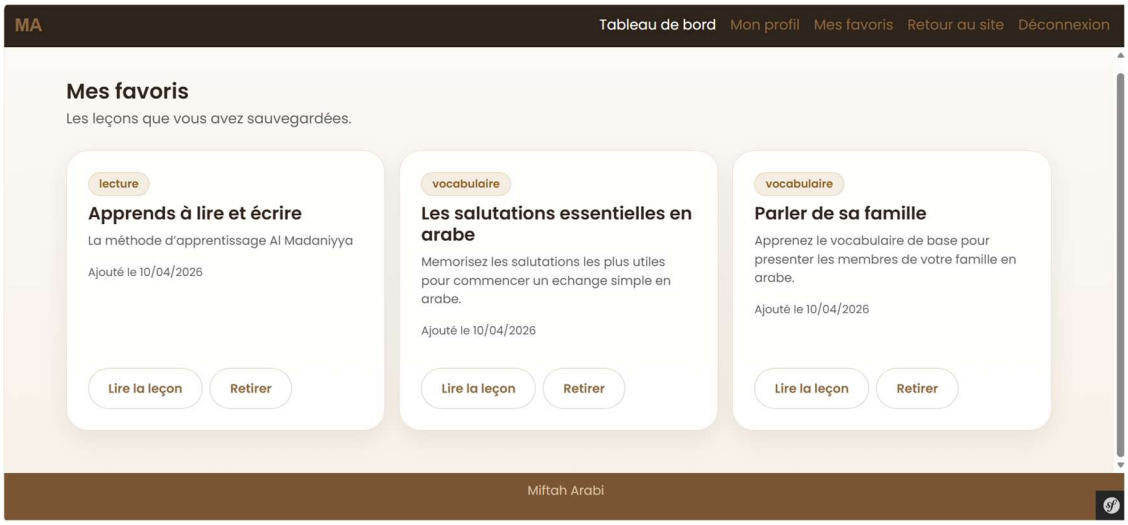


Figure 18 : Liste des leçons favorites

7.3 Espace administration

L'espace d'administration permet :

- D'accéder à un tableau de bord ;
- De gérer les catégories ;
- De gérer les tags ;
- De gérer les leçons ;
- De publier et dépublier les leçons ;
- De gérer les ressources associées ;
- De consulter les messages de contact ;
- De gérer les utilisateurs et leurs rôles ;
- De gérer le profil administrateur.

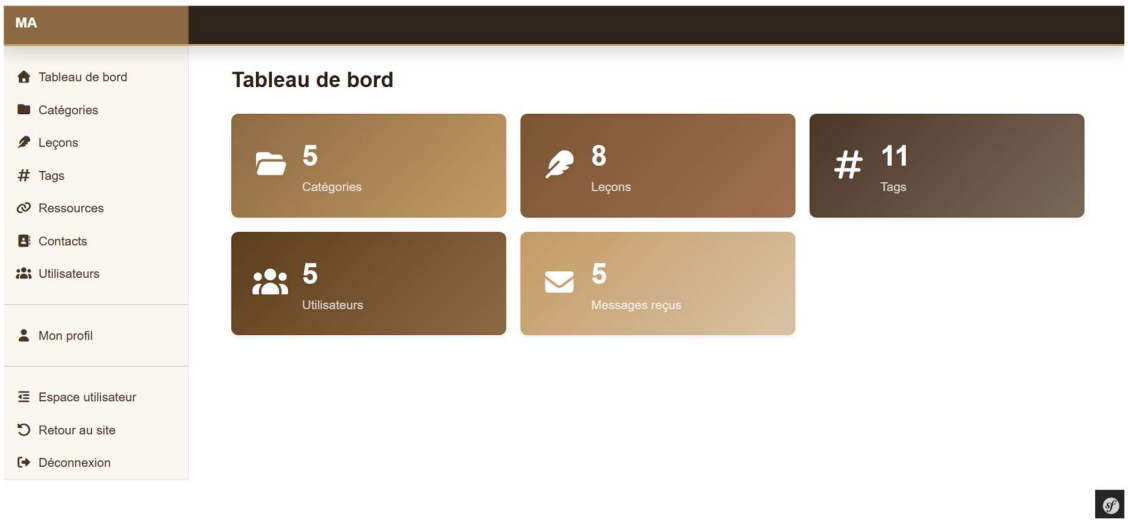


Figure 19 : Tableau de bord administrateur

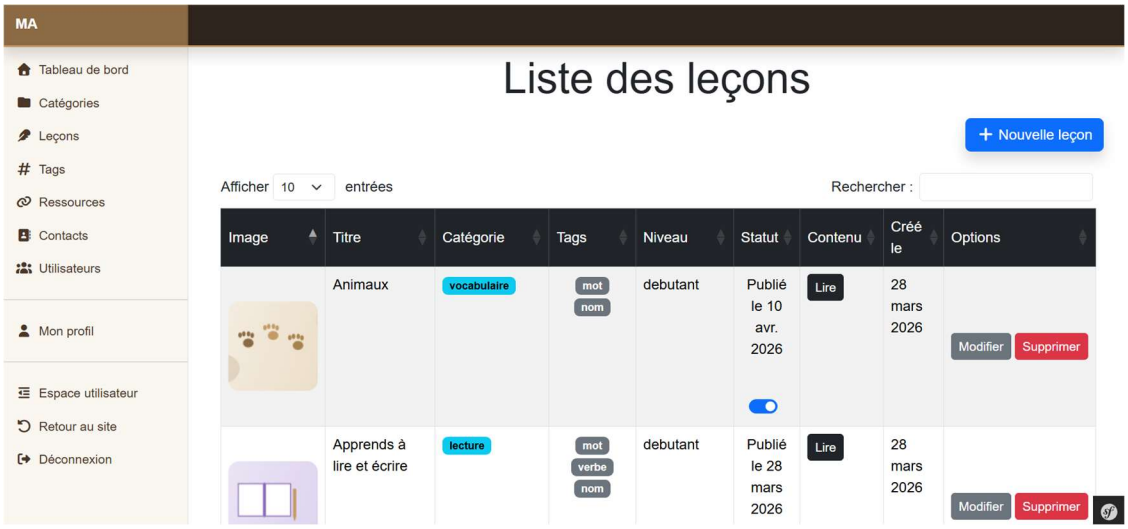


Figure 20 : Gestion des leçons côté administration

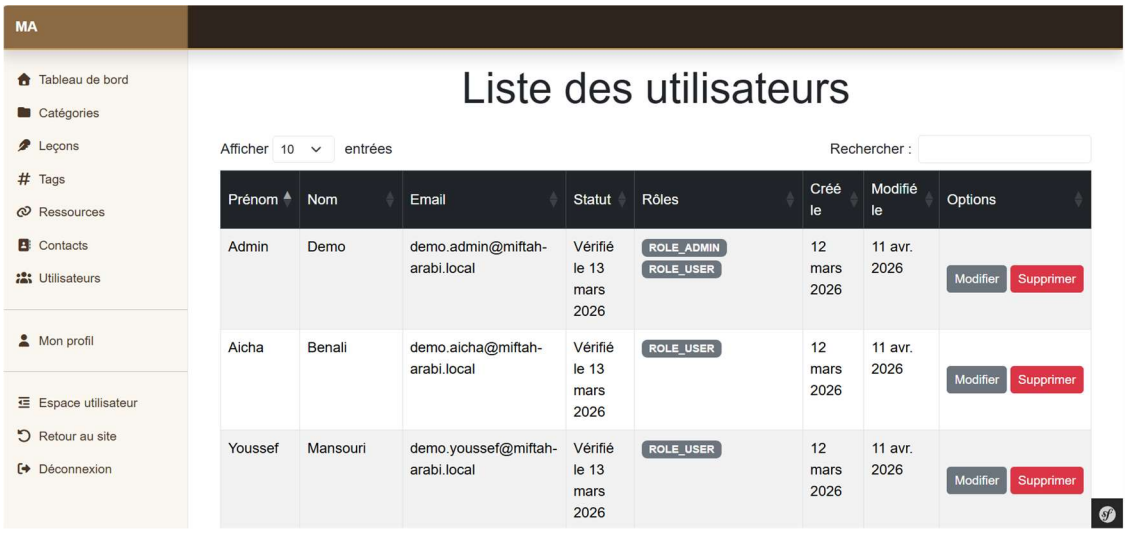


Figure 21 : Gestion des utilisateurs côté administration

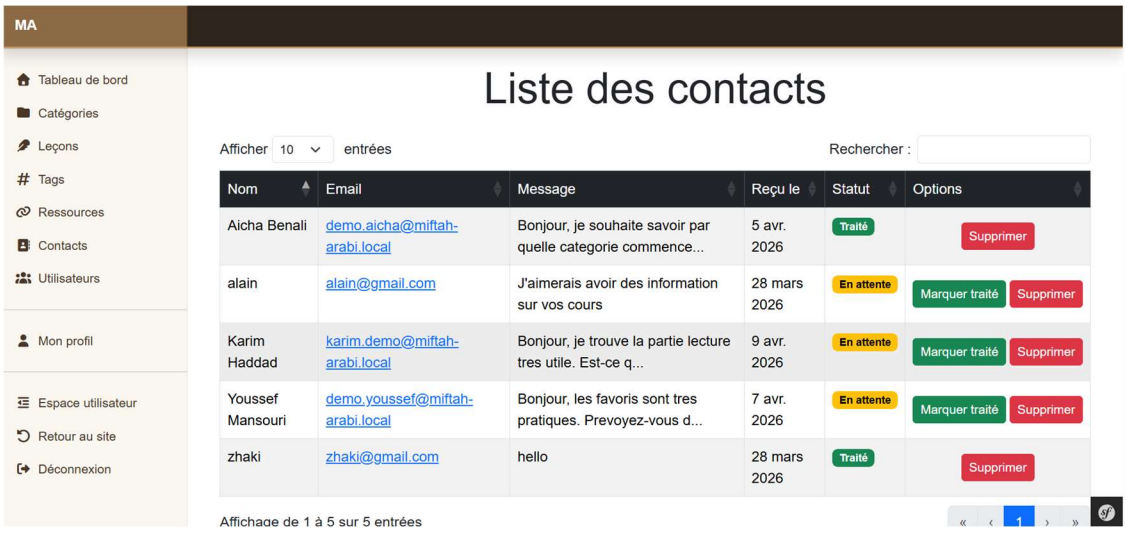


Figure 22 : Gestion des messages de contact côté administration

7.4 Mode maintenance

Le projet intègre un mode maintenance piloté par commande Symfony. Lorsqu'il est activé, un fichier var/maintenance.lock est créé et un EventSubscriber intercepte les requêtes pour afficher une page dédiée.

Cette fonctionnalité répond à un besoin réel d'exploitation et montre une approche plus professionnelle du cycle de vie applicatif.

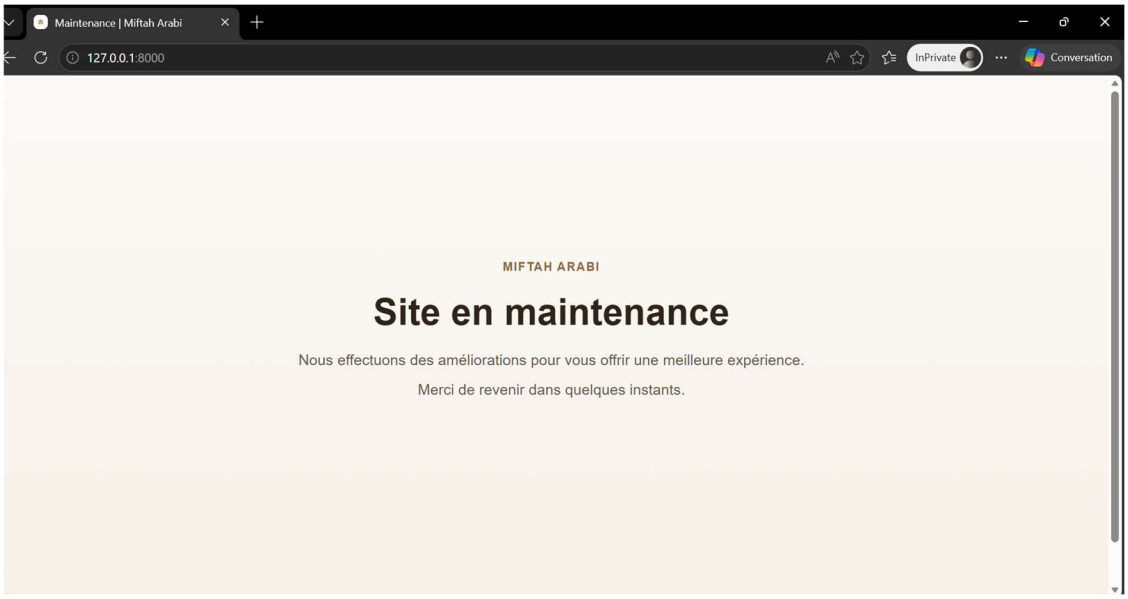


Figure 23 : Page de maintenance

7.5 SEO et pages annexes

Le site intègre également plusieurs éléments utiles à la visibilité et au cadre légal :

- robots.txt
- Sitemap
- Balises SEO dans les templates
- Pages mentions légales
- Page conditions générales d'utilisation
- Page plan du site
- Pages d'erreur personnalisées 403 / 404

Figure 24 : Page des mentions légales

Figure 25 : Page des conditions générales d'utilisation

Figure 26 : Plan du site XML généré par l'application

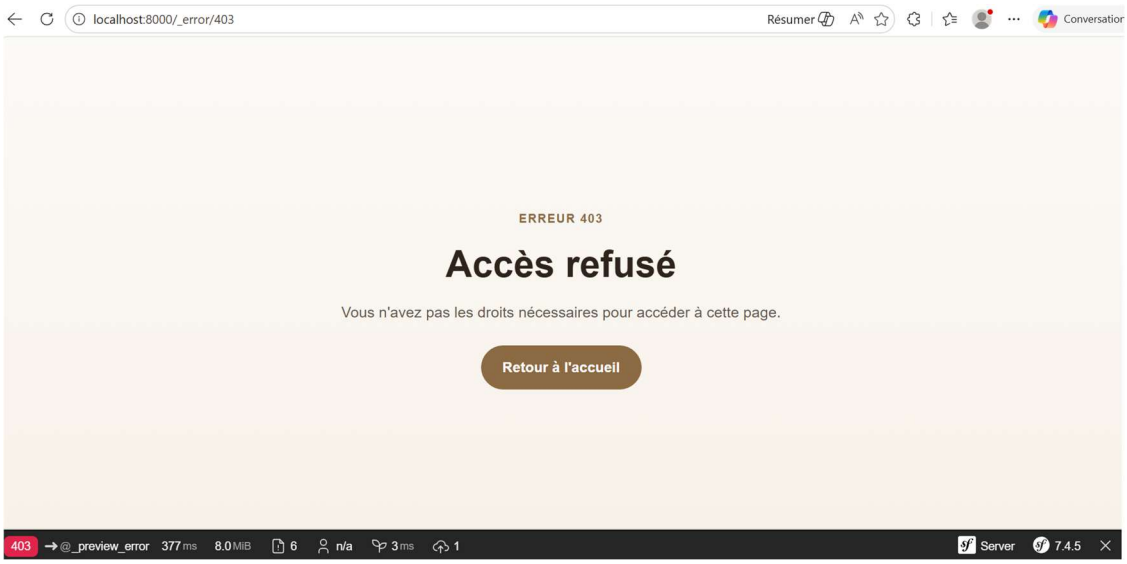


Figure 27 : Page d'erreur 403 personnalisée

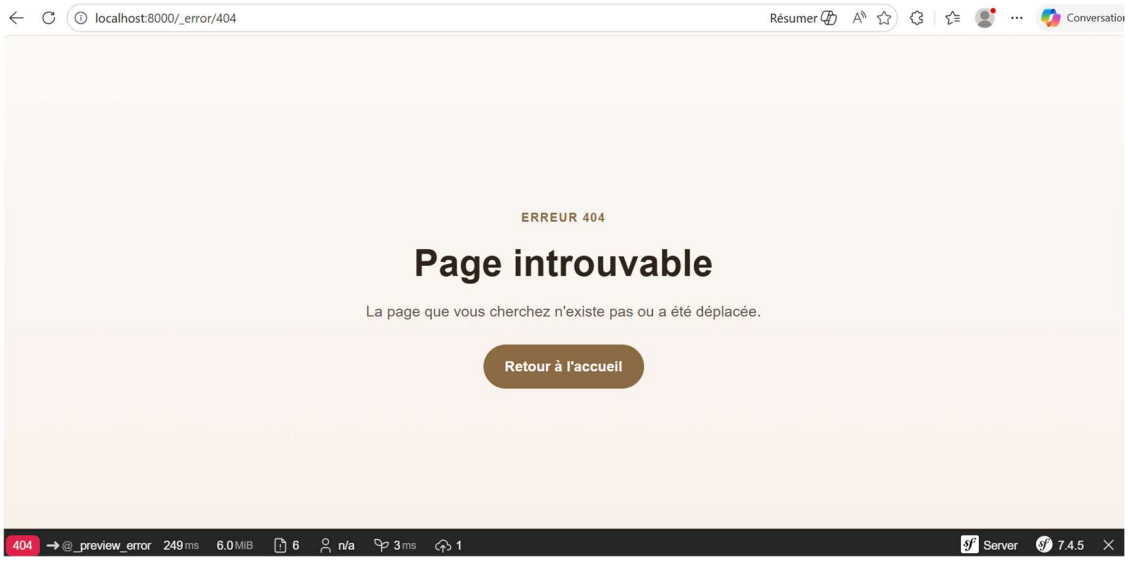


Figure 28 : Page d'erreur 404 personnalisée

---

8. Sécurité

La Sécurité a été une préoccupation constante pendant le projet.

8.1 Contrôle des accès

Le fichier de Sécurité Symfony protège les zones sensibles :

- /admin est réservé aux administrateurs ;
- /user est réservé aux utilisateurs authentifiés.

L'authentification s'appuie sur un custom_authenticator, un user provider basé sur l'email et un UserChecker.

8.2 Sécurité des mots de passe

Les mots de passe sont hachés avec le composant de Sécurité Symfony. Les règles de validation imposent :

- Une longueur minimale de 12 caractères ;
- Une majuscule ;
- Une minuscule ;
- Un chiffre ;
- Un caractère spécial.

8.3 Vérification d'email

L'inscription n'est pas suffisante pour accéder au compte : l'utilisateur doit confirmer son adresse email. Cette étape réduit le risque de comptes frauduleux ou non maitrises.

8.4 Protection CSRF

Les actions sensibles comme l'ajout ou la suppression utilisent des jetons CSRF. C'est le cas notamment dans l'administration et dans la gestion des favoris.

8.5 Protection contre les injections

Le projet bénéficie des mécanismes natifs de Symfony et Doctrine :

- Échappement automatique de Twig contre les XSS ;
- Formulaires Symfony avec validation côté serveur ;
- Requêtes Doctrine paramétrées contre les injections SQL.

8.6 Sécurisations des uploads

Les images de leçons sont gérées via **VichUploaderBundle** avec :

- Validation du type de fichier ;
- Limite de taille ;
- Nommage unique ;
- Suppression automatique lors des mises à jour ou suppressions.

8.7 Limitation des tentatives de connexion

La connexion est protégée par un mécanisme de login_throttling configuré a **3 tentatives par minute**, ce qui limite la force brute sur le formulaire de connexion.

8.8 Maintenance et administration

Le mode maintenance et la commande de création du super administrateur participent aussi à la maîtrise technique du projet, même si certaines améliorations restent possibles.

9. Veille technologique

Tout au long du projet, j'ai réalisé une veille ciblée sur :

- La Sécurité Symfony ;
- Les bundles utilisés ;
- Les bonnes pratiques de développement ;
- Les problématiques d'authentification et de gestion des uploads.

9.1 Sources utilisées

- Documentation officielle Symfony ;
- Documentation officielle des bundles utilisés ;
- Documentation Doctrine ;
- Documentation MDN ;
- GitHub et changelogs des dépendances ;
- Ressources ANSSI, OWASP et articles de référence.

9.2 Exemples de sujets suivis

- Vérification d'email et authentification sécurisée ;
- Réinitialisation du mot de passe ;
- Failles classiques OWASP ;
- Audit des dépendances avec `composer audit` ;
- Bonnes pratiques autour de la gestion des secrets et fichiers.`env`.

9.3 Utilité concrète de la veille

Cette veille m'a aidé a :

- Résoudre certaines difficultés techniques ;
- Comprendre les bonnes pratiques de Sécurité ;
- Mieux justifier mes choix d'implémentation ;
- Identifier des axes d'amélioration pour la production.

10. Jeu d'essai

Les tests ont été réalisés principalement de manière **manuelle** à partir de scénarios fonctionnels. En pratique, j'ai beaucoup testé moi-même les parcours utilisateur et administrateur tout au long du développement, souvent après chaque fonctionnalité importante. Le projet contient PHPUnit dans ses dépendances de développement, mais il n'y a pas encore de vraie couverture automatisée. J'assume ce point comme une limite actuelle du projet.

10.1 données de test utilisées

Comptes

- Un compte administrateur ;
- Un compte utilisateur vérifié ;
- Un compte utilisateur non vérifié pour tester les restrictions de connexion.

Contenus

- Plusieurs catégories ;
- Plusieurs tags ;
- Des leçons publiées et non publiées ;
- Des ressources vidéo ;
- Des messages de contact ;
- Des favoris utilisateur.

10.2 Scénarios testés

Navigation visiteur

Action	Résultat attendu	Résultat obtenu
Ouvrir la page d'accueil	Affichage des dernières leçons publiées	OK
Ouvrir la liste des leçons	Pagination et affichage corrects	OK
Filtrer par catégorie, tag ou niveau	Résultats filtres	OK
Consulter une leçon	Contenu et ressources visibles	OK
Envoyer un message de contact	Message enregistre et confirmation affichée	OK

Authentification

Action	Résultat attendu	Résultat obtenu
Inscription avec données invalides	Erreurs de validation	OK
Inscription valide	Création du compte et email envoyé	OK
Connexion sans email vérifié	Refus de connexion	OK
Réinitialisation du mot de passe	Email reçu et mot de passe modifié	OK

Espace utilisateur

Action	Résultat attendu	Résultat obtenu
Modifier son profil	Données mises à jour	OK
Ajouter une leçon aux favoris	Leçon visible dans la liste des favoris	OK
Retirer une leçon des favoris	Leçon supprimée de la liste	OK

Administration

Action	Résultat attendu	Résultat obtenu
Accéder a /admin sans être administrateur	Accès refuse	OK
Créer une catégorie	Catégorie enregistrée	OK
Créer une leçon	Leçon enregistrée	OK
Publier / dépublier une leçon	Statut modifié	OK
Ajouter une ressource à une leçon	Ressource visible sur la leçon	OK
Consulter les messages de contact	Messages affiches dans l'admin	OK
Activer le mode maintenance	Page maintenance visible	OK

---

11. Difficultés rencontrées

11.1 Vérification d'email

L'une des premières difficultés importantes a concerné le fonctionnement combiné de la vérification d'email et du UserChecker. Au début, je ne comprenais pas toujours clairement à quel moment le compte devait être considéré comme actif ou bloqué. J'ai dû faire plusieurs essais, relire le code, revoir le cours correspondant et tester les emails avec Mailtrap avant d'arriver à un fonctionnement correct.

11.2 Upload d'images

La configuration de l'upload avec VichUploaderBundle a demandé plusieurs ajustements : mapping, configuration du formulaire, validation des fichiers, emplacement de destination et comportement de mise à jour. C'est une partie sur laquelle j'ai perdu du temps, parce qu'une petite erreur de configuration suffisait à bloquer tout le parcours d'ajout ou de modification d'image.

11.3 Sécurité Git et fichiers sensibles

J'ai commité par erreur des fichiers d'environnement au début du projet. Cette erreur m'a obligé à nettoyer l'historique Git et à formaliser un .env.example. Sur le moment, c'était stressant, mais cela m'a appris concrètement à mieux protéger les secrets et à contrôler mon .gitignore avant de pousser mon travail.

11.4 Évolution du périmètre

Le projet final a évolué au-delà du MVP initial. Au fur et à mesure, j'avais envie d'ajouter des fonctionnalités utiles, mais il fallait aussi rester raisonnable pour pouvoir terminer le projet proprement. Cette gestion du périmètre n'a pas toujours été simple, surtout dans le cadre d'une formation où l'on apprend encore en même temps qu'on développe.

11.5 Écart entre théorie et production

Le projet est complet fonctionnellement, mais j'ai aussi compris l'écart qu'il peut y avoir entre un projet de formation réussi et une vraie mise en production totalement finalisée. Certaines dimensions restent à renforcer avant une exploitation réelle :

- Mise en production documentée de bout en bout ;
- Automatisation de tests ;
- Exploitation en conditions réelles ;
- Capture et formalisation des preuves finales pour le jury.

11.6 Correctifs de fiabilisation apportés en phase finale

En phase de recette finale, j'ai identifié et corrigé plusieurs points qui ne bloquaient pas l'usage courant mais qui auraient pu poser des problèmes en conditions réelles.

Contrôle d'autorisation sur la suppression des favoris

Un utilisateur connecté pouvait potentiellement tenter de supprimer un favori qui ne lui appartenait pas si l'identifiant était connu. J'ai ajouté une vérification côté serveur pour m'assurer que le favori appartient bien à l'utilisateur authentifié avant toute suppression.

Contrainte d'intégrité sur les favoris

Le code applicatif évitait déjà les doublons de favoris, mais la base de données ne les interdisait pas formellement. J'ai ajouté une contrainte unique sur le couple `user_id` / `lesson_id` en base, avec une migration de nettoyage préalable. Cette double protection garantit l'intégrité à la fois côté applicatif et côté base.

Protection de la suppression d'une catégorie référencée

Une catégorie pouvait être supprimée alors que des leçons y étaient encore rattachées, ce qui aurait provoqué une incohérence métier. J'ai bloqué la suppression dans ce cas et ajouté un message explicite dans l'interface d'administration.

Correction du flux dans la commande CreateSuperAdmin

La commande console de création du super administrateur échouait de façon incorrecte lorsque des utilisateurs existaient déjà en base. J'ai corrigé le flux pour que la commande échoue uniquement si un super administrateur existe déjà, et poursuive normalement dans les autres cas.

---

12. Bilan et perspectives

12.1 Bilan

Au terme du projet, j'arrive à présenter une application web pédagogique complète, structurée et exploitable, avec :

- Un front-office public ;
- Un espace utilisateur ;
- Un back-office ;
- Une base de données relationnelle ;
- Une authentification sécurisée ;
- Une gestion du contenu complète ;
- Une organisation Git cohérente par branches et pull requetés ;
- Une documentation de projet déjà avancée.

Je ne considère pas ce projet comme “parfait”, mais comme un vrai projet de fin de formation, sur lequel j’ai beaucoup progressé techniquement et méthodologiquement. Il montre que je suis capable de partir d’un besoin, de concevoir une solution, de la développer et de la faire évoluer jusqu’à une version présentable.

12.2 Ce que j’ai appris

Ce projet m’a permis de progresser sur :

- La conception de base de données ;
- La structuration MVC dans Symfony ;
- La sécurité web ;
- La gestion des formulaires et de l’authentification ;
- Le versioning Git/GitHub ;
- La gestion de projet en sprints ;
- La documentation technique d’un projet complet.

Il m’a aussi appris à être plus patient dans ma façon de travailler. J’ai compris qu’en développement, il faut souvent revenir sur ses étapes, corriger, tester, recommencer et accepter de ne pas réussir du premier coup.

12.3 Ce que j’améliorerais

Avec plus de temps, je renforcerais en priorité :

- La couverture de tests automatisés ;
- Certaines protections métier côté base de données ;
- La documentation de mise en production ;
- La finition de quelques détails UX et validation.

Si je devais refaire ce projet depuis le début, je passerais aussi encore plus de temps sur la stratégie de tests et sur la préparation du déploiement final, pour réduire les ajustements de dernière minute.

12.4 Perspectives

Les évolutions possibles du projet sont :

• Mise en place d'un nom de domaine personnalisé ;

• Couverture de tests automatisés avec PHPUnit (tests unitaires et fonctionnels) ;

• Suivi de progression individuel pour les apprenants ;

• Quiz et exercices interactifs associés aux leçons ;

• Commentaires sur les leçons ;

• Évolution vers une plateforme e-learning complète.

13. Annexes

Annexe A - Compétences DWWM mobilisées

Le projet mobilise directement les compétences suivantes du titre professionnel :

- Installer et configurer son environnement de travail ;
- Maquetter des interfaces utilisateur ;
- Réaliser des interfaces statiques ;
- Développer la partie dynamique des interfaces ;
- Mettre en place une base de données relationnelle ;
- Développer des composants d'accès aux données ;
- Développer des composants métier côté serveur ;
- Documenter le déploiement d'une application web.

Annexe B - Structure du projet

```
projet-miftah-arabi/
|-- assets/
|-- config/
|-- migrations/
|-- public/
|-- src/
|   |-- Command/
|   |-- Controller/
|   |-- Entity/
|   |-- EventSubscriber/
|   |-- Form/
|   |-- Repository/
|   `-- Security/
|-- templates/
|-- tests/
`-- documentation/
```

Annexe C - Captures techniques complémentaires

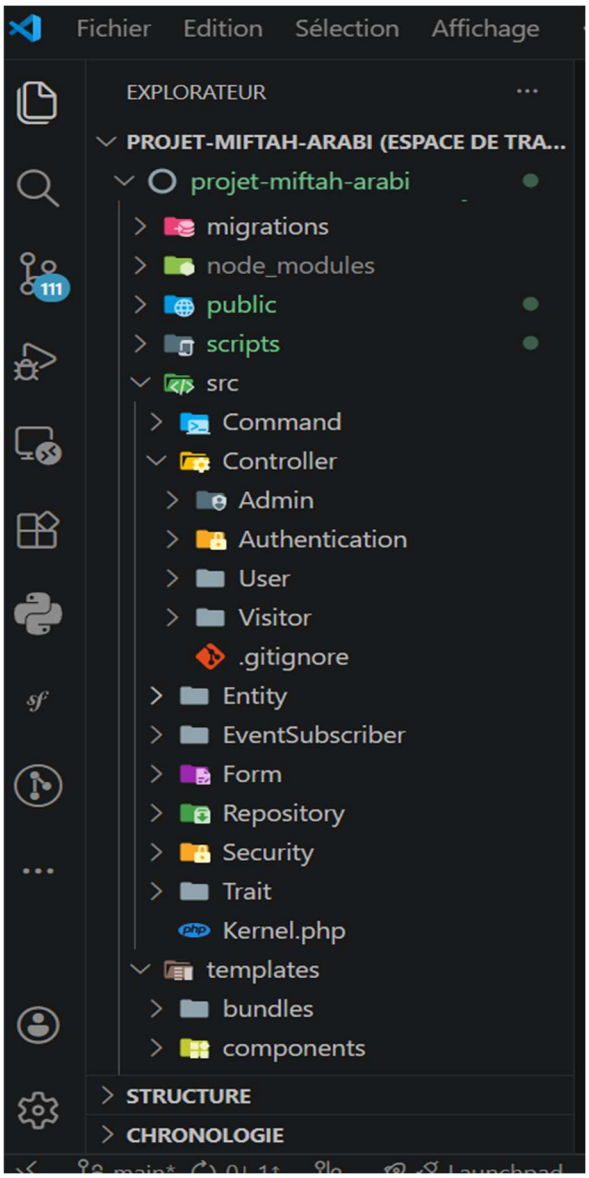


Figure 29 : Arborescence du projet Symfony dans VS Code

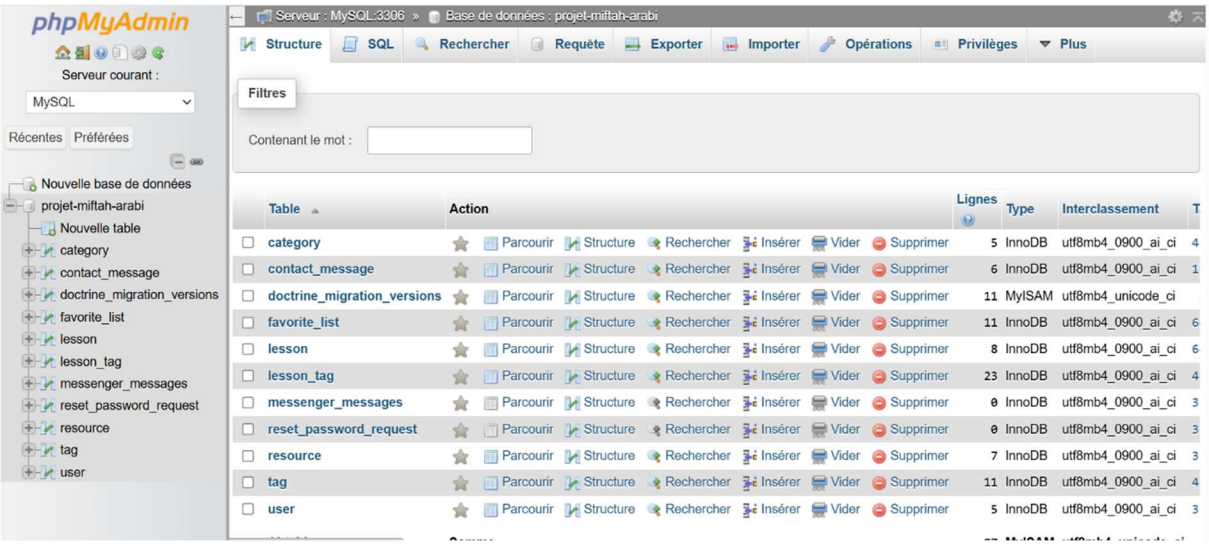


Figure 30 : Tables de la base de données dans phpMyAdmin

Figure 31 : Affichage responsive mobile de la page d'accueil